Blade Rush

Réalisé par:

Akrame CHAKOUK
Hicham MOHAMMED
Walid KSOURI

Soutenance de projet

Made with GAMMA

Plan de la présentation

1

1. Présentation du Jeu

Inspiration, concept, contexte et objectifs.

2

2. Architecture & Choix Techniques

Architecture logicielle, choix technologiques.

3

3. Gameplay & Intelligence Artificielle

Minimax & Alpha-Beta, exécution, optimisations, complexité et résultats.

4

4. Gestion de Projet & Bilan

Diagramme de Gantt, gestion de projet et bilan.

5

5. Conclusion

Réflexions finales et ouverture.

Made with GAMMA

Présentation du Jeu

Concept

Duel 1v1 à l'épée inspiré de Nidhogg, progression dans l'arène

Gameplay

Déplacements horizontaux + saut, affrontement en temps réel

Système de Combat

Positions haute-médiane-basse, désarmement

Objectif

Progresser à travers plusieurs zones jusqu'à la base ennemie

Made with GAMMA

Architecture Logicielle

SDL3

Bibliothèque multiplateforme Fournit le rendu 2D, le clavier, la fenêtre et le timingAvantage principal : abstraction complète des entrées et du renduRésultat : logique du jeu totalement indépendante de SDL

Séparation des responsabilités

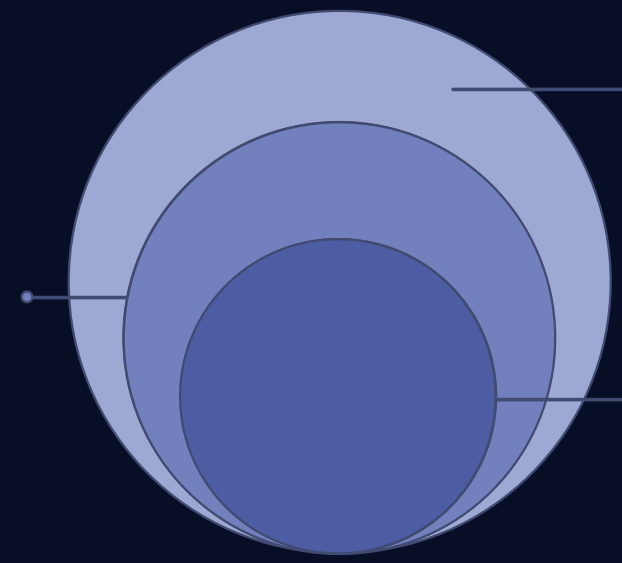
La logique du jeu reste indépendante de SDL et ne dépend que de ses entrées abstraites.

Couche bas niveau

SDL3 gère la fenêtre, le rendu 2D, le clavier et le timing.

Made with GAMMA

Architecture Logicielle



Séparation des responsabilités

→ **engine**

SDL, fenêtre, input, temps

→ **game**

Physique, règles, rooms

→ **ai**

Décision – indépendante du renderer

→ **render**

Dessin + caméra uniquement

Made with GAMMA

Le Coeur du Moteur: Le Fichier Main

Le fichier `main.c` est le point d'entrée minimaliste de notre application, orchestrant l'initialisation et la boucle principale du jeu.

Phase d'Initialisation

```c
Game game;
RenderContext render;
FrameInput input = {0};

EngineConfig config;
f64 accumulator = 0.0;
u64 last_counter;

memory_init();

config.title = WINDOW_TITLE;
config.window_width = WINDOW_WIDTH;
config.window_height = WINDOW_HEIGHT;

engine = engine_create(&config);
if (!engine) {
    log_error("engine_create failed");
    return 1;
}

if (!render_init(&render, engine)) {
    log_error("render_init failed");
    engine_destroy(engine);
    return 1;
}

if (!game_init(&game)) {
    log_error("game_init failed");
    render_shutdown(&render);
    engine_destroy(engine);
    return 1;
}
```

La Boucle Principale

```c
last_counter = engine_now_counter();

while (engine_is_running(engine)) {
    FrameInput sampled_input = {0};
    u64 now = engine_now_counter();
    f64 frame_time = engine_counter_seconds(now - last_counter);
    last_counter = now;

    if (frame_time > 0.25) {
        frame_time = 0.25;
    }

    accumulator += frame_time;

    engine_poll_input(engine, &sampled_input);
    input_accumulate_frame(&input, &sampled_input);

    if (input.quit_requested || game_is_quit_requested()) {
        engine_request_stop(engine);
    }

    while (accumulator >= FIXED_DT) {
        game_update(&game, &input, FIXED_DT);
        accumulator -= FIXED_DT;
        input_clear_edges(&input);
    }

    render_frame(&render, engine, &game);
}
```

```c
game_shutdown(&game);
render_shutdown(&render);
engine_destroy(engine);
```

Made with GAMMA

Engine

```
typedef struct EngineConfig {
    const char* title;
    i32 window_width;
    i32 window_height;
} EngineConfig;
```

```
struct Engine {
    SDL_Window* window;
    SDL_Renderer* renderer;
    bool running;
    PlayerKeybind keybinds[MAX_PLAYERS];
    FrameInput previous_input;
};
```

```
void engine_poll_input(Engine* engine, FrameInput* out_input
{
```

```
{
    const bool* keys = SDL_GetKeyboardState(NULL);
    engine_apply_keyboard_to_frame_input(engine, keys, out_input);
}

engine->previous_input = *out_input;
```

Made with GAMMA

Le Module Game

Le module `game` est le cœur logique de Blade Rush, définissant le monde du jeu et ses interactions.

Logique du Jeu

Comprend l'arène, les mécanismes de combat, les états des joueurs et l'intelligence artificielle.

Règle Fondamentale

Le gameplay **ne voit jamais SDL**. Cette séparation garantit une portabilité et une maintenabilité accrues.

Actions Abstraites

Il manipule uniquement des actions abstraites liées aux joueurs, dissociées de toute implémentation matérielle directe.

Responsabilité du Moteur

C'est le moteur (engine) qui est chargé de traduire les touches réelles du clavier en ces actions abstraites.

Cette architecture garantit que la logique du jeu reste pure et réutilisable, indépendamment des spécificités techniques de l'affichage ou de la gestion des entrées.

Made with GAMMA

Game

```c
void game_update(Game* game, const FrameInput* input, f32 dt)
{
    PlayerCommand p1_cmd;
    PlayerCommand p2_cmd;
    MenuAction menu_action;

    if (!game || !input) {
        return;
    }

    if (game->phase == GAME_PHASE_MENU) {
        menu_update(&game->menu, input, dt);
        menu_action = menu_get_action(&game->menu);

        switch (menu_action) {
```

```c
game_collect_commands(game, input, &p1_cmd, &p2_cmd, dt);

combat_step(&game->combat, &game->arena, &p1_cmd, &p2_cmd, dt);
game_update_room_push_from_kill(game);

if (game->combat.round_over) {
    game_handle_round_end(game);
    return;
}

game_handle_room_transitions(game);
```

Made with GAMMA

Le Module Render

Le module de rendu est une entité distincte et spécialisée, essentielle pour la clarté et la robustesse de notre architecture.

Lecture de l'État du Jeu

Il se contente de lire l'état actuel du jeu, sans jamais tenter de le modifier, puis l'affiche via la bibliothèque SDL.

Isolation du Gameplay

Le module Render n'intervient jamais dans la logique du gameplay. Cette isolation garantit que les actions des joueurs et les règles du jeu restent intactes et prévisibles.

Déterminisme et Propreté

Cette séparation stricte assure un moteur de jeu propre et un comportement parfaitement déterministe côté logique, facilitant le débogage et la maintenabilité.

Made with GAMMA

Render

```
void render_frame(RenderContext* render, Engine* engine, const Game* game)
{
    SDL_Renderer* renderer;
    const Room* room;
```

```
render_update_camera(render, game);

render_draw_room(render, engine, room);
render_draw_fighters(render, engine, game);
render_draw_thrown_sword(render, engine, &game->combat);
```

Made with GAMMA

Génération Procédurale des Arènes

Pour garantir une expérience de jeu dynamique et rejouable, le jeu utilise une approche procédurale pour construire ses arènes.

Chaque **arène** est conçue comme une séquence de **salles (rooms)** générées dynamiquement. Ce processus assure une *variété* constante à chaque partie, évitant la monotonie et encourageant l'adaptation stratégique du joueur.

La génération est basée sur un principe d'**aléatoire contrôlé**, où des règles définies guident la création pour maintenir l'équilibre et la jouabilité, tout en introduisant des éléments imprévisibles.

Avantages

- Gain de temps de développement
- Variété de gameplay infinie
- Économie de mémoire (pas de maps pré-conçues)

Inconvénients

- Peut manquer de créativité manuelle
- Risque de répétitivité dans les motifs
- Nécessite des règles de génération robustes

Made with GAMMA

Minimax + Alpha-Beta

Pourquoi Minimax ?

Jeu adversarial à deux agents aux objectifs opposés. L'état continu est **discrétisé** – l'IA raisonne sur des instants de décision, pas à chaque frame SDL.

Espace d'actions (b ≈ 8)

Fonction d'évaluation

score = $+w_1 \times$ sword_advantage
$+ w_2 \times$ attack_opportunity
$- w_3 \times$ edge_danger
$- w_4 \times$ bad_distance

Alpha-Beta élague les branches sans changer le résultat théorique de Minimax.

- Avancer / Reculer
- Sauter
- Thrust
- Hauteur épée H/M/B
- Lancer épée

Made with GAMMA

Exécution de l'Algorithme Minimax

1. Génération de l'arbre

À chaque tour, on génère un arbre de tous les coups possibles à partir de la position actuelle. Chaque branche correspond à une action ou un coup possible.

2. Feuilles de l'arbre

Les feuilles représentent les positions finales ou évaluées. Si le jeu est terminé, la valeur est connue (+1, 0, -1). Sinon, on applique une fonction d'évaluation. Aux feuilles, il n'y a pas d'action, seulement une valeur.

3. Propagation des valeurs avec Minimax

On remonte les valeurs des feuilles vers la racine. MAX (ordinateur) choisit le maximum, tandis que MIN (joueur) choisit le minimum.

4. Choix de l'action

Chaque action du premier niveau reçoit une valeur calculée. L'ordinateur choisit l'action avec la meilleure valeur, c'est-à-dire la plus grande.

Ce processus permet à l'IA d'explorer les conséquences possibles d'un coup avant de décider, en tenant compte à la fois de l'optimisation de son propre score et de la réaction adverse.

Made with GAMMA

Optimisations Pratiques de Minimax

En pratique, reconstruire entièrement l'arbre à chaque coup serait trop lent pour des jeux complexes comme les échecs ou le Go. Pour permettre une décision en temps réel, il faut donc s'appuyer sur des optimisations essentielles.

Élagage alpha-bêta

Élimine les branches inutiles dès qu'on sait qu'elles ne seront pas choisies.

Mémorisation / table de transposition

On réutilise les sous-arbres déjà calculés.

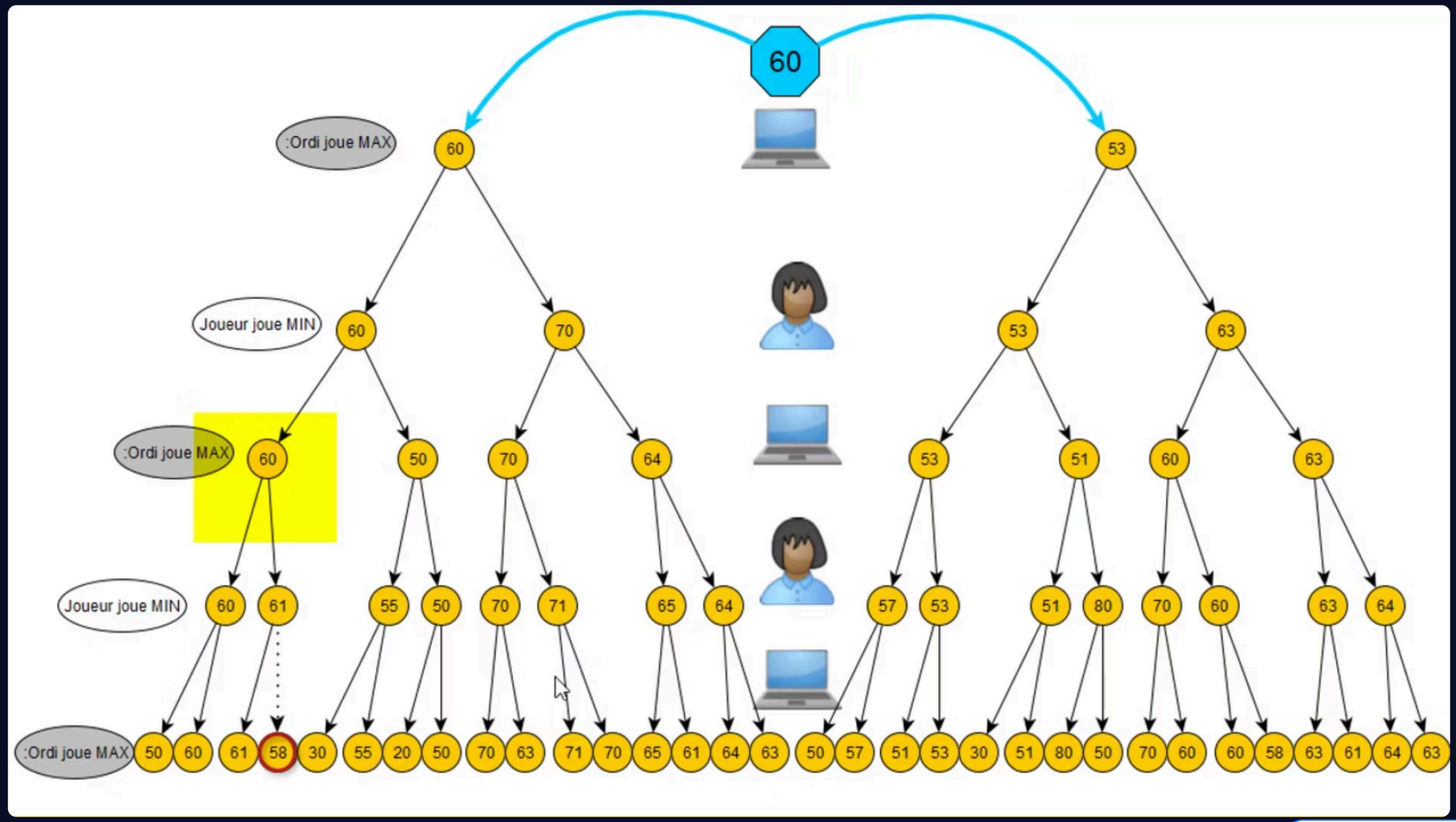
Profondeur limitée + heuristiques

On ne calcule pas tous les coups possibles, seulement ceux qui semblent pertinents.

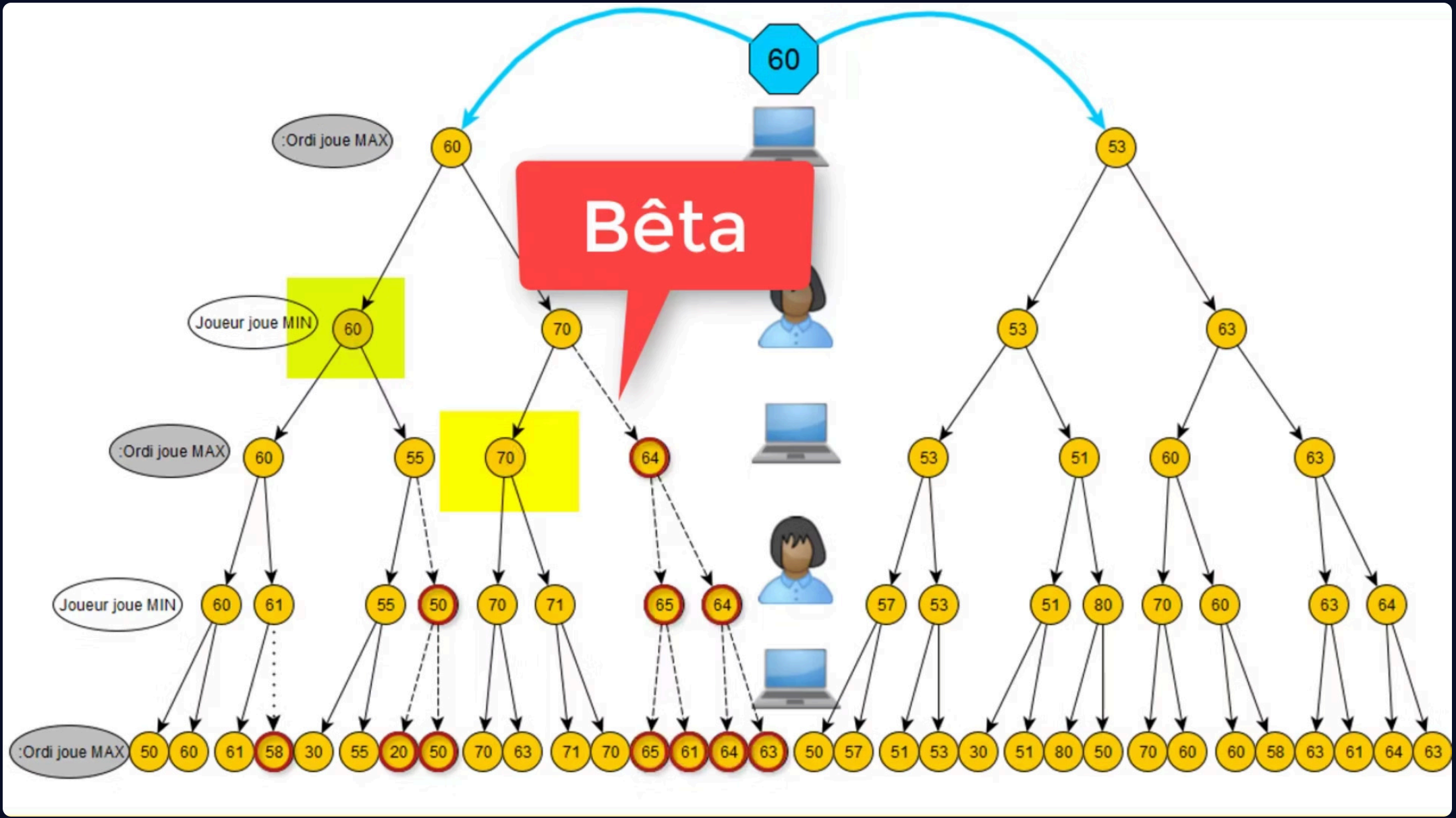
Ces optimisations sont critiques pour rendre Minimax suffisamment rapide et réactif dans un contexte de jeu en temps réel.

Made with GAMMA

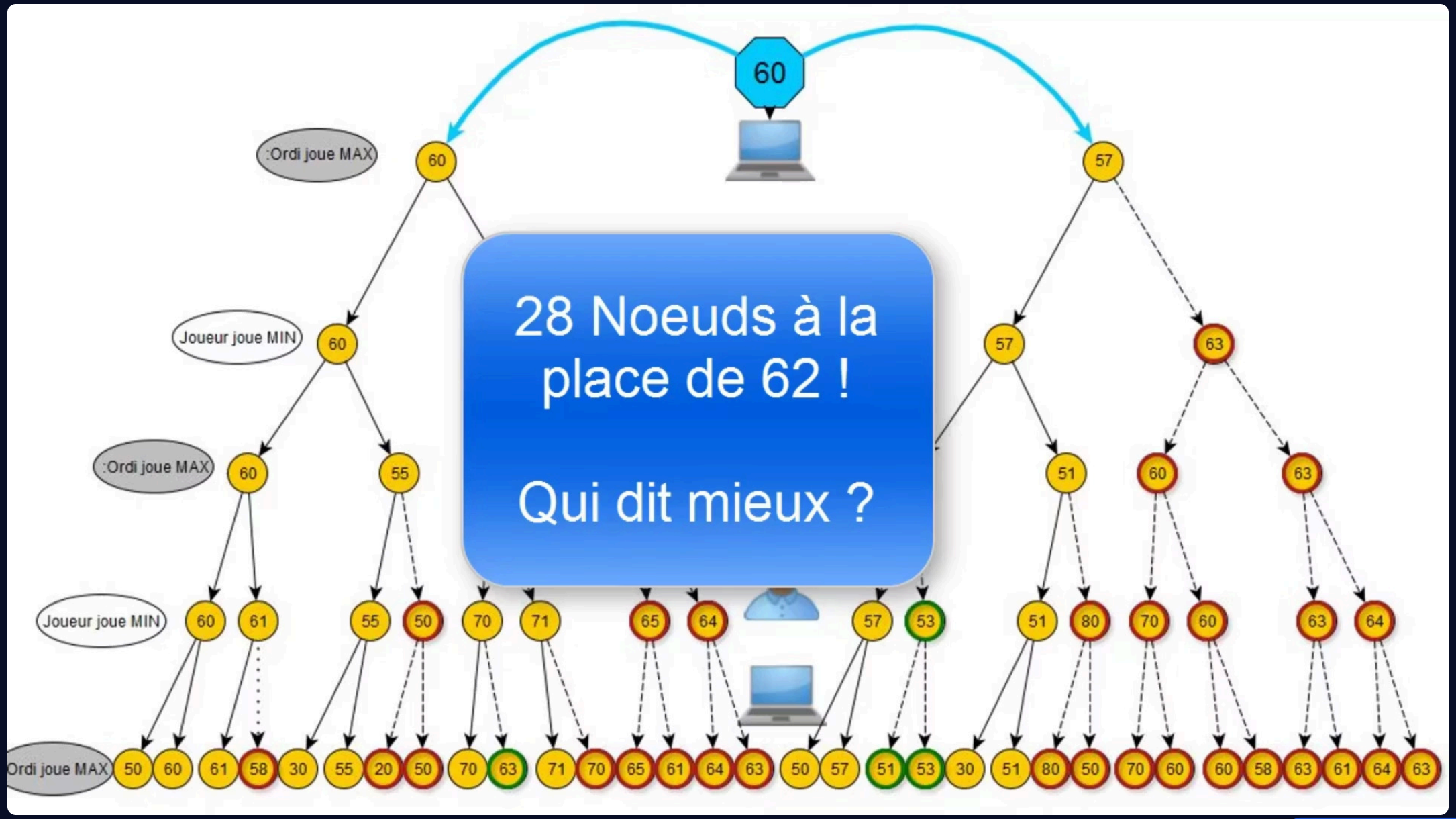
Élagage alpha-bêta



Made with GAMMA



Made with GAMMA



Made with GAMMA

Complexité Théorique

Temps — Minimax

O(b^d) – croissance exponentielle

Temps — Alpha-Beta

Meilleur cas : fortement réduit
Pire cas : identique à Minimax

Mémoire

Pile récursive → **O(d)**

Paramètres critiques

b

Facteur de branchement (~8 actions)

d

Profondeur de recherche – paramètre le plus sensible

Le paramètre critique est **d** : le coût augmente exponentiellement avec la profondeur.

Made with GAMMA

Evaluation

Difficulté EASY

```
[INFO] AI[P2] sample=9 depth=2 complexity=O(8^d) full_tree=73 nodes=72 leaves=64 cutoffs=0 prune=1.4% think=0.0
23ms mem=3.4KB action_idx=0
[INFO] AI[P2] sample=10 depth=2 complexity=O(8^d) full_tree=73 nodes=72 leaves=64 cutoffs=0 prune=1.4% think=0.
027ms mem=3.4KB action_idx=1
[INFO] AI[P2] sample=11 depth=2 complexity=O(8^d) full_tree=73 nodes=72 leaves=64 cutoffs=0 prune=1.4% think=0.
019ms mem=3.4KB action_idx=0
[INFO] AI[P2] sample=12 depth=2 complexity=O(8^d) full_tree=73 nodes=72 leaves=64 cutoffs=0 prune=1.4% think=0.
026ms mem=3.4KB action_idx=0
[INFO] AI[P2] sample=13 depth=2 complexity=O(8^d) full_tree=73 nodes=72 leaves=64 cutoffs=0 prune=1.4% think=0.
021ms mem=3.4KB action_idx=1
[INFO] AI[P2] sample=14 depth=2 complexity=O(8^d) full_tree=73 nodes=72 leaves=64 cutoffs=0 prune=1.4% think=0.
024ms mem=3.4KB action_idx=0
[INFO] AI[P2] sample=15 depth=2 complexity=O(8^d) full_tree=73 nodes=72 leaves=64 cutoffs=0 prune=1.4% think=0.
022ms mem=3.4KB action_idx=0
```

Difficulté EXPERT

```
[INFO] AI[P2] sample=39 depth=5 complexity=O(8^d) full_tree=37449 nodes=4164 leaves=2895 cutoffs=964 prune=88.9
% think=1.101ms mem=195.2KB action_idx=2
[INFO] AI[P2] sample=40 depth=5 complexity=O(8^d) full_tree=37449 nodes=3536 leaves=2413 cutoffs=880 prune=90.6
% think=0.952ms mem=165.8KB action_idx=0
[INFO] AI[P2] sample=41 depth=5 complexity=O(8^d) full_tree=37449 nodes=4164 leaves=2895 cutoffs=964 prune=88.9
% think=0.642ms mem=195.2KB action_idx=2
[INFO] AI[P2] sample=42 depth=5 complexity=O(8^d) full_tree=37449 nodes=3536 leaves=2413 cutoffs=880 prune=90.6
% think=0.940ms mem=165.8KB action_idx=0
[INFO] AI[P2] sample=43 depth=5 complexity=O(8^d) full_tree=37449 nodes=4164 leaves=2895 cutoffs=964 prune=88.9
% think=1.112ms mem=195.2KB action_idx=2
[INFO] AI[P2] sample=44 depth=5 complexity=O(8^d) full_tree=37449 nodes=3536 leaves=2413 cutoffs=880 prune=90.6
% think=0.815ms mem=165.8KB action_idx=0
[INFO] AI[P2] sample=45 depth=5 complexity=O(8^d) full_tree=37449 nodes=4164 leaves=2895 cutoffs=964 prune=88.9
% think=1.098ms mem=195.2KB action_idx=2
[INFO] AI[P2] sample=46 depth=5 complexity=O(8^d) full_tree=37449 nodes=3536 leaves=2413 cutoffs=880 prune=90.6
% think=0.709ms mem=165.8KB action_idx=0
[INFO] AI[P2] sample=47 depth=5 complexity=O(8^d) full_tree=37449 nodes=4164 leaves=2895 cutoffs=964 prune=88.9
% think=1.078ms mem=195.2KB action_idx=2
```

Made with GAMMA

GitHub

Lien du projet

Scannez le QR code pour accéder au dépôt GitHub avec le code source complet et les instructions d'installation.

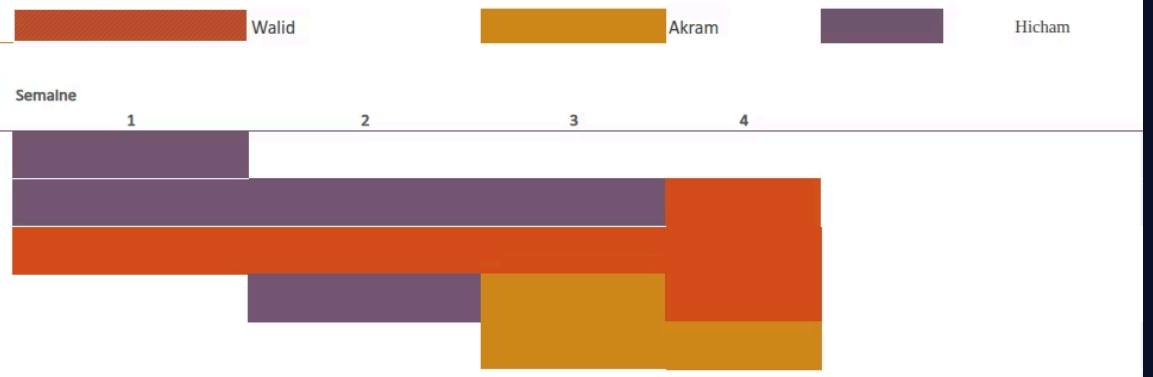
Made with GAMMA

Diagramme de Gantt du Projet

Chaque tâche couvre une semaine, avec une progression séquentielle .

Digramme de Gantt

ACTIVITE	DÉBUT PRÉVU	DÉBUT RÉEL	DURÉE RÉELLE	DURÉE RÉELLE	POURCENTAGE D'AVANCEMENT
S1 — Architecture	1	5	1	4	75%
S2 — Moteur + rendu	1	6	1	6	100%
S3 — Combat MVP	2	4	2	5	75%
S4 — Rooms / transitions	4	8	4	6	90%
S5 — IA Minimax	4	2	4	8	85%
S6 — Mesures & bilan	4	3	4	6	85%



Made with GAMMA

Gestion de Projet & Bilan

Équipe & Rôles

Hicham Mohammed

Architecture & moteur

AKRAME CHAKOUK

Gameplay & arene

Mohamed Walid KSOURI

IA & analyse

Ce qui est réussi

- Prototype fonctionnel
- Architecture modulaire
- IA Minimax opérationnelle
- Mesures expérimentales obtenues

Limites & Perspectives

- État discret simplifié
- Profondeur contrainte par le temps réel

Made with GAMMA

Conclusion

1

Succès du Projet

Développement d'un moteur de jeu avec architecture modulaire, logique de gameplay indépendante, et une IA fonctionnelle.

- Moteur opérationnel
- Gameplay principal présent
- IA Minimax fonctionnelle

2

Base Solide

Malgré l'absence d'assets visuels avancés ou de son, la base technique est robuste et offre une plateforme stable pour l'avenir.

- Architecture modulaire
- Logique déterministe
- Facilite le débogage

3

Perspectives Futures

Le projet est prêt à être enrichi avec de nouvelles fonctionnalités et améliorations, notamment l'IA.

- Ajouter animations et son
- Étendre le gameplay
- Améliorer l'IA

Améliorations Clés de l'IA

- Rework de la fonction d'évaluation avec plus de critères.
- Affinement du modèle décisionnel.
- Optimisation des heuristiques pour un comportement plus crédible et compétitif.

Ce projet ambitieux pose une fondation solide sur laquelle il est possible de continuer à construire.

Made with GAMMA

Merci beaucoup !

Des questions ?

Made with GAMMA